Natural Language Processing

Class 10: LLMs: Modeling, data, and tokenization

Adam Faulkner

March 31, 2026

1. Introduction
2. Solving NLP tasks via prompt-based learning
3. Sampling strategies during LLM generation
4. Data
5. LLM Pretraining
6. Tokenization

1 Introduction

2 Solving NLP tasks via prompt-based learning

3 Sampling strategies during LLM generation

4 Data

5 LLM Pretraining

6 Tokenization

The Large Language Model revolution

- LLMs leverage a very basic insight: knowledge of language, context, and the world can be learned by being taught to predict the next word, again and again, based on context, in a (very) large corpus of text
- This is called **pretraining**: learning knowledge about language and the world from iteratively predicting tokens in vast amounts of text

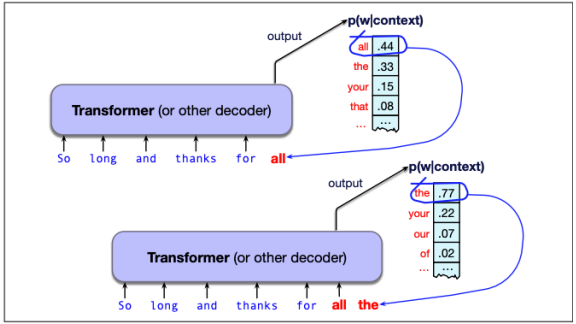
The Large Language Model revolution

- What can language models learn from word prediction? Consider the examples below. What kinds of knowledge do you think the model might pick up from learning to predict which word fills the underbar (the correct answer is shown in blue)?

With roses, dahlias, and peonies, I was surrounded by _____ flowers
The room wasn't just big it was _____ enormous
The square root of 4 is _____ 2
The author of "A Room of One's Own" is _____ Virginia Woolf
The professor said that _____ he

The Large Language Model revolution

- During pretraining, we use the decoder-Transformer architecture discussed in a previous class to turn a predictive model that gives a probability distribution over next words into a generative model by repeatedly sampling from the distribution
- The result is a left-to- right (also called autoregressive) language model. As each token is generated, it gets added onto the context as a prefix for generating the next token.



The prompt-based learning revolution

- With the release of GPT-3, LLM researchers introduced a revolutionary new way of solving NLP problems traditionally solved using traditional machine learning: **prompt-based learning**
- Rather than training individual models —either via traditional methods or BERT-based finetuning—one can solve NLP tasks by simply providing text to an LLM trained via causal language modeling task and ask the LLM to condition on this task to solve the task

The prompt-based learning revolution

- Language models that have been instruction-tuned are very good at following instructions, answering questions and carrying on a conversation and can be prompted
- A **prompt** is a text string that a user issues to a language model to get the model to do something useful
- In prompting, the users prompt string is passed to the language model, which iteratively generates tokens conditioned on the prompt
- The process of finding effective prompts for a task is known as **prompt engineering**

The prompt-based learning revolution

- When a single test instance is given at prompt-time we call this setting **zero-shot**; when one or more examples are provided to the LLM to illustrate the task, we call this setting **few-shot**.
- Consider the following few-shot example from the MMLU dataset

The following are multiple choice questions about high school computer science.

Let x = 1. What is x << 3 in Python 3?
(A) 1 (B) 3 (C) 8 (D) 16
Answer: C

Which is the largest asymptotically?
(A) O(1) (B) O(n) (C) O(n^2) (D) O(log(n))
Answer: C

What is the output of the statement "a" + "ab" in Python 3?
(A) Error (B) aab (C) ab (D) a ab
Answer:

In-context learning via prompts

- Prompts can be viewed as a learning signal. This is especially clear when a prompt has demonstrations, since the demonstrations can help language models learn to perform novel tasks from these examples of the new task
- This kind of learning is different than the gradient descent-based learning that occurs in pretraining
- The weights of the model are not updated by prompting; what changes is just the context and the activations in the network
- This is called **in-context learning**

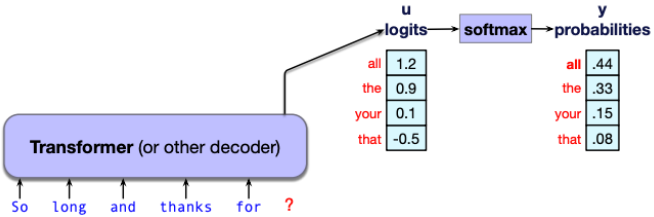
Prompt structure

- Prompts generally have static and dynamic sections
- The static section is called the set of **system instructions**
- The system instructions define the task or role for the LLM, and sets overall tone and context.
- The system prompt is silently prepended to any user text. Here are the first few instructions from Claude's system instructions

```
Claude should give concise responses to very simple questions,
but provide thorough responses to complex and open-ended
questions.
Claude is able to explain difficult concepts or ideas clearly.
It can also illustrate its explanations with examples, thought
experiments, or metaphors.
Claude does not provide information that could be used to
make chemical or biological or nuclear weapons
For more casual, emotional, empathetic, or advice-driven
conversations, Claude keeps its tone natural, warm, and
empathetic
Claude cares about people's well-being and avoids encouraging
or facilitating self-destructive behavior
If Claude provides bullet points in its response, it should
use markdown, and each bullet point should be at least 1-2
sentences long unless the human requests otherwise
```

Prompt structure

- The generation depends on the probability of each token
- The internal networks for language models (whether transformers or alternatives like LSTMs or state space models) generate scores called logits for each token in the vocabulary. This score vector u is then normalized by softmax to be a legal probability distribution



1 Introduction

2 Solving NLP tasks via prompt-based learning

3 Sampling strategies during LLM generation

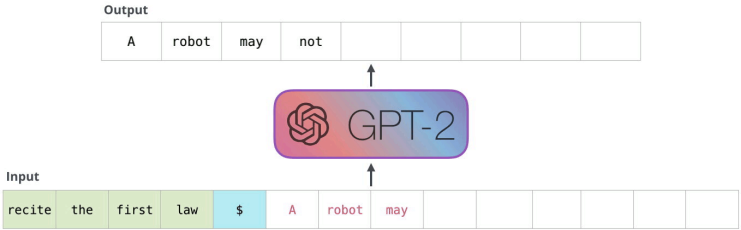
4 Data

5 LLM Pretraining

6 Tokenization

The autoregressive language modeling task

- As we learned in a previous class, GPT (Generative Pretrained Transformer)-style models are instances of **causal** language modeling: these models predict the next token in a sequence of tokens, and the model can only attend to tokens to its left.
- These models are trained on an *autoregressive language modeling task*: after each token is produced, that token is added to the sequence of inputs and that new sequence becomes the input to the model in its next step



Can all NLP tasks be solved using next-word prediction?

- Causal LLMs, particularly the GPT family of LLMs, introduced the novel idea that **all NLP tasks could be cast as word prediction tasks**.
- Another framing: All NLP tasks can be recast as cases of **conditional generation**.
- Conditional generation is the task of generating text conditioned on an input piece of text.
- The idea leverages the concept of a **prompt**, a piece of input text that is fed to an LLM, which then continues generating text token by token, conditioned on the prompt.

Classification as text completion

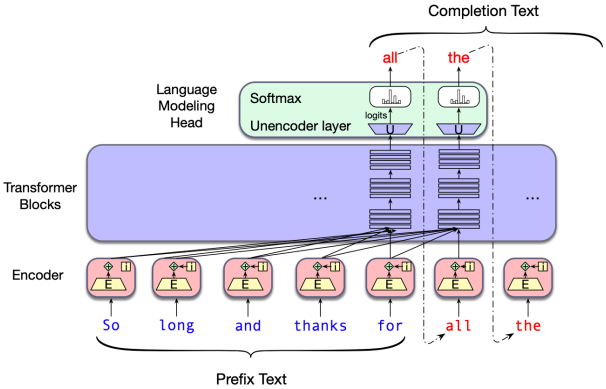


Figure 1: Left-to-right (also called autoregressive) text completion with transformer-based LLMs. As each token is generated, it gets added to the context as a prefix for generating the next token.

Classification as word prediction: Sentiment analysis

- For example, we can cast sentiment analysis as a text-completion task by giving a language model a context like:

```
The sentiment of the sentence "The movie was great" is:
```

- We let the LLM complete the sentence and then evaluate the following conditional probability of the words "positive" and the word "negative" to see which is higher:

$$P(\text{positive}|\text{The sentiment of the sentence "The movie was great" is:})$$

$$P(\text{negative}|\text{The sentiment of the sentence "The movie was great" is:})$$

Question-answering as word prediction

- We can cast the task of question-answering as word prediction by giving an LLM a question and a token such as `A:` suggesting that an answer should come next:

```
Q: Who wrote the book The Origin of Species"?  A:
```

- We then ask a language model to compute the probability distribution over possible next words given this prompt:

$$P(w|\text{Q: Who wrote the book ”The Origin of Species”? A:})$$

- We then let the LLM continue predicting words, with each word adding to the existing context until we get the answer: `Charles Darwin`

$$P(\text{Charles}|\text{Q: Who wrote the book ”The Origin of Species”? A:})$$

$$P(\text{Darwin}|\text{Q: Who wrote the book The Origin of Species"? A: Charles})$$

1 Introduction

2 Solving NLP tasks via prompt-based learning

3 Sampling strategies during LLM generation

4 Data

5 LLM Pretraining

6 Tokenization

The importance of sampling strategies

- The most common method for decoding in large language models is **sampling**
- Sampling from a model's distribution over words means choosing random words according to their probability assigned by the model
- A good sampling strategy will find the optimal tradeoff between **quality** and **diversity**
- Methods that emphasize the most probable words tend to produce generations that are rated by people as more accurate, more coherent, and more factual, but also more boring and more repetitive
- Methods that give a bit more weight to the middle-probability words tend to be more creative and more diverse, but less factual and more likely to be incoherent or otherwise low-quality

Generating the next token via sampling

- The task of choosing a token to generate based on the models probabilities is often called **decoding**
- We decode from an LLM in a left-to-right manner (or right-to-left for languages like Arabic or Hebrew) by sampling
- We discussed various sampling strategies in a previous class – let's review the most popular form of LLM sampling, **temperature sampling**

Temperature sampling

- Divide the logits by a temperature parameter τ before we normalize it by passing it through the softmax
- Higher values of τ lead to greater variability, more creative text
- Lower values of τ lead to boring, generic text
- Why this works:
  - Softmax pushes high values toward 1 and low values toward 0
  - The lower τ is, the larger the scores being passed to the softmax since dividing by a smaller fraction ≤ 1 results in making each score larger
  - When larger numbers are passed to a softmax the result is a distribution with increased probabilities of the most high-probability words and decreased probabilities of the low probability words

Temperature sampling

softmax output with temperature τ

	logits	close to greedy τ=0.1	close to greedy τ=0.5	normal softmax τ=1	close to uniform τ=10	close to uniform τ=100
all	1.2	.95	.59	.44	.27	.25
the	0.9	.05	.32	.33	.26	.25
your	0.1	0	.07	.15	.24	.25
that	-0.5	0	.02	.08	.23	.25

low temperature sampling (towards greedy)

high temperature sampling (towards uniform)

Temperature sampling

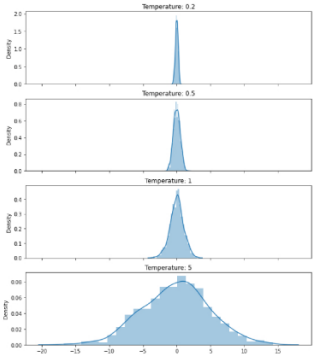


Figure 2: Temperature sampling: As the temperature nears 0, the probability of the most likely word approaches 1.

1 Introduction

2 Solving NLP tasks via prompt-based learning

3 Sampling strategies during LLM generation

4 Data

5 LLM Pretraining

6 Tokenization

Data used in LLM pretraining

- Web text is usually taken from corpora of automatically-crawled web pages like the **common crawl**, a series of snapshots of the entire web produced by the nonprofit Common Crawl (`https://commoncrawl.org/`) that each have billions of webpages.
- Various versions of common crawl data exist, such as the Colossal Clean Crawled Corpus (C4), a corpus of 156 billion tokens of English that is filtered in various ways (deduplicated, removing non-natural language like code, sentences with offensive words from a blocklist).

Data used in LLM pretraining

The Pile: an 825 GB English text corpus that is constructed by publicly released code, containing again a large amount of text scraped from the web as well as books and Wikipedia

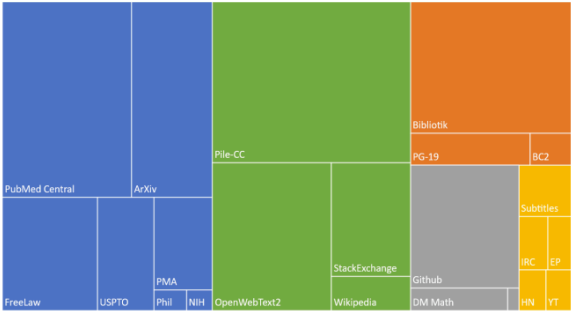


Figure 3: The Pile corpus, color coded as academic, internet, prose (books), dialogue (movie subtitles, chat data), and misc..

1 Introduction

2 Solving NLP tasks via prompt-based learning

3 Sampling strategies during LLM generation

4 Data

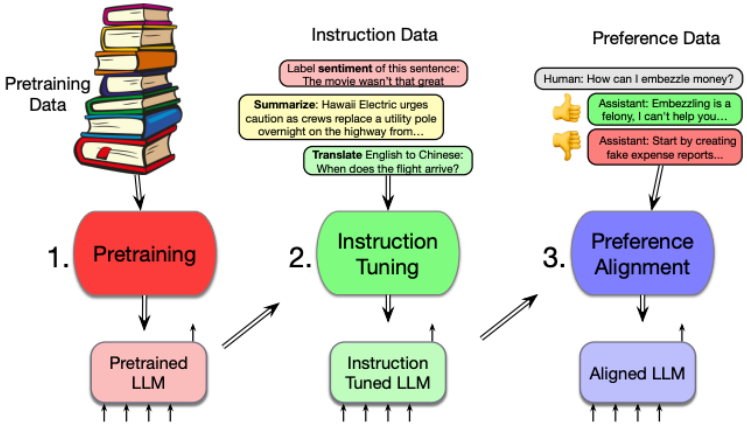
5 LLM Pretraining

6 Tokenization

The LLM learning objective

- In self-training for language modeling, we take a corpus of text as training material and at each time step t ask the model to predict the next word.
- At first it will do poorly at this task, but since in each case we know the correct answer (it's the next word in the corpus!) over time it well get better and better at predicting the correct next word – this is the **self-supervised** learning setting
- In the next class we'll describe the **instruction tuning** and **preference alignment** steps of LLM development

The LLM learning objective



The LLM learning objective

- The weight matrices for the decoder Transformer's feedforward and attention components are trained by error backpropagation with gradient descent
- We use the cross-entropy loss function to minimize and pass back through the network: this measures the difference between a predicted probability distribution and the correct distribution. The probability distribution is over the token vocabulary:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$

The LLM learning objective

- In the case of language modeling, the correct distribution y_t comes from knowing the next word. This is represented as a one-hot vector corresponding to the vocabulary where the entry for the actual next word is 1, and all the other entries are 0
- Thus, the cross-entropy loss for language modeling is determined by the probability the model assigns to the correct next token
- So without loss of generality we can say that at time t the cross-entropy loss in the above equation can be simplified as the negative log probability the model assigns to the next word in the training sequence, and $logp(w_t + 1)$, or more formally, using $\hat{y}$ to mean the vector of estimated token probabilities from the language model:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

The LLM learning objective

- At each step, given all the preceding tokens, the language model produces an output distribution over the entire vocabulary. During training, the probability assigned to the correct word is used to calculate the cross-entropy loss for each item in the sequence. The loss for each batch is the average cross-entropy loss over the entire sequence of negative log probabilities, or more formally:

$$L_{CE}(\text{batch of length T}) = \frac{1}{T}\sum_{t=1}^{T} -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

- The weights in the network are then adjusted to minimize this average cross-entropy loss over the batch via gradient descent using error backpropagation

1 Introduction

2 Solving NLP tasks via prompt-based learning

3 Sampling strategies during LLM generation

4 Data

5 LLM Pretraining

6 Tokenization

Tokenization

- Tokenization has also been key to the development of contemporary LLMs.
- Our early discussion of **ngram models** introduced us to the most basic form of tokenization: splitting on whitespace.
- But this is the least efficient form of tokenization and doesn't address the **out-of-vocabulary** problem: what if, during inference, the model encounters an **ngram** that it hasn't seen during training?
- Tokenization schemes such as **byte-pair-encoding** and **WordPiece** (introduced by BERT's developers) address these issues,

Tokenization: Byte-Pair Encoding

- Instead of defining tokens as words or as characters (as in Chinese), we can use our data to automatically tell us what the tokens should be.
- A better tokenization algorithm will also help us address the **out-of-vocabulary** (OOV) problem, i.e, when an LM encounters a word that it hasn't seen during training.
- To deal with this OOV errors modern tokenizers automatically induce sets of tokens that include tokens smaller than words, called **subwords**

Tokenization: Byte-Pair Encoding

- Contemporary tokenization algorithms such as **Byte-pair encoding** and **WordPiece** have two parts: a **token learner**, and a **token segmenter**
- The token learner takes a raw training corpus (sometimes roughly preseparated into words, for example by whitespace) and induces a vocabulary, a set of tokens.
- The token segmenter takes a raw test sentence and segments it into the tokens in the vocabulary

Tokenization: Byte-Pair Encoding

- The simplest and most widely used tokenization algorithm is **byte-pair encoding**, which works as follows:

1. Start with a vocabulary that is just the set of all individual characters (the alphabet, numbers, etc.).
2. Examining the training corpus, choose the two symbols that are most frequently adjacent (say 'A','B')
3. The new merged symbol 'AB' is added to the vocabulary
4. Every adjacent 'A' B' in the corpus is replaced with the new AB.
5. Steps 2 - 4 are repeated, creating new longer and longer character strings, until k merges have been done creating k novel tokens; k is thus a parameter of the algorithm.

Tokenization: Byte-Pair Encoding

An example using an input corpus of 18 word tokens with counts for each word (the word *low* appears 5 times, the word *newer* 6 times, and so on), which would have a starting vocabulary of 11 letters:

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

Tokenization: Byte-Pair Encoding

The BPE algorithm first counts all pairs of adjacent symbols: the most frequent is the pair `e r` because it occurs in *newer* (frequency of 6) and *wider* (frequency of 3) for a total of 9 occurrences. We then merge these symbols, treating `er` as one symbol, and count again:

corpus		vocabulary
5	l o w _	_, d, e, i, l, n, o, r, s, t, w, er
2	l o w e s t _	
6	n e w er _	
3	w i d er _	
2	n e w _	

Tokenization: Byte-Pair Encoding

Now the most frequent pair is `er __`, which we merge; our system has learned that there should be a token for word-final `er`, represented as `er __`:

corpus

5	l o w _
2	l o w e s t _
6	n e w er_
3	w i d er_
2	n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_

Tokenization: Byte-Pair Encoding

Next `n e` (total count of 8) gets merged to `ne`:

corpus	
5	l o w _
2	l o w e s t _
6	ne w er_
3	w i d er_
2	ne w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_, ne

Tokenization: Byte-Pair Encoding

- Our final vocabulary for this tiny corpus is

  `_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_`

- Once we've learned our vocabulary, the token segmenter is used to tokenize a test sentence. The token segmenter just runs the merges we have learned from the training data on the test data.
- By the end, if the test data contains the character sequence `n e w e r _`, it would be tokenized as a full word since we have `newer_` in our vocabulary
- But the characters of a new OOV word like `l o w e r_` would be merged into the two tokens `low er _`

Next class: April 14

No class next week April 7

- Next: Aligning LLMs to human preferences and instructions
- Reading
  - Jurafsky & Martin Chapter 9: Post-training: Instruction Tuning, Alignment, and Test-Time Compute
  - Training language models to follow instructions with human feedback
  - Direct Preference Optimization: Your Language Model is Secretly a Reward Model
  - Mixtral of Experts